SR UNIVERSITY

AI Assisted Coding. **2403A51L40 [B-52]**

Assignment 15.2 **Akhila**

Task 1: Setup FastAPI Backend (VS Code)

Install:

pip install fastapi uvicorn

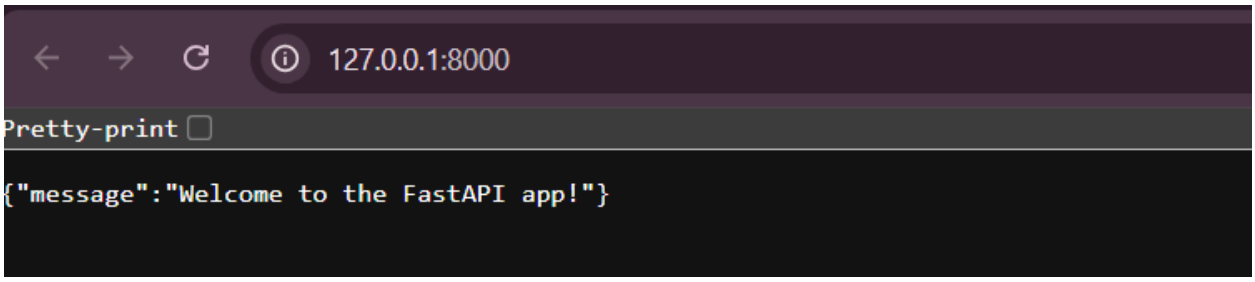
Prompt: Create a basic FastAPI app with one GET endpoint "/" that returns a welcome message.

Code:

```
from fastapi import FastAPI
app = FastAPI()
@app.get("/")
async def read_root():
    return {"message": "Welcome to the FastAPI app!"}
```

Run: python -m uvicorn main:app --reload

Output: http://127.0.0.1:8000/



Explanation: This code sets up a basic FastAPI application with endpoints to create, read, update, and delete items in a list, along with docstrings for Swagger documentation.

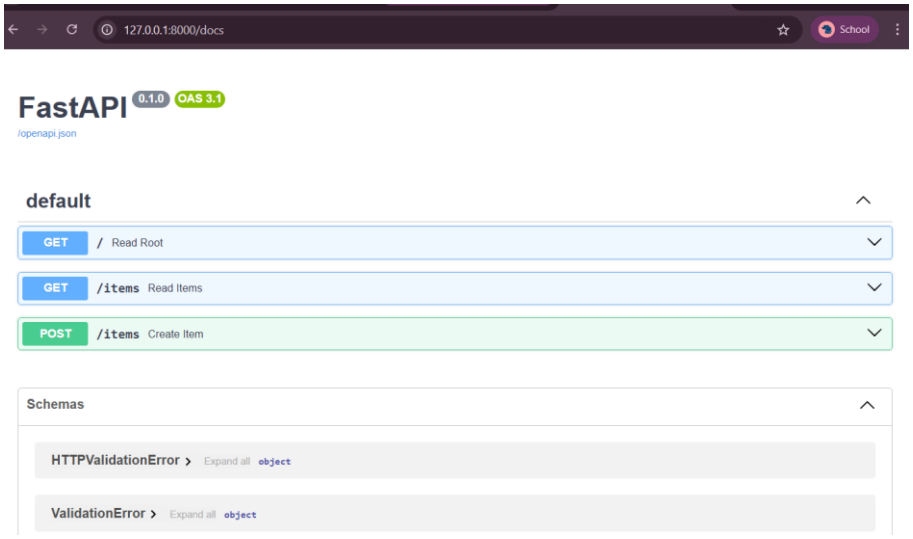
Task 2: API – Create and Read Items

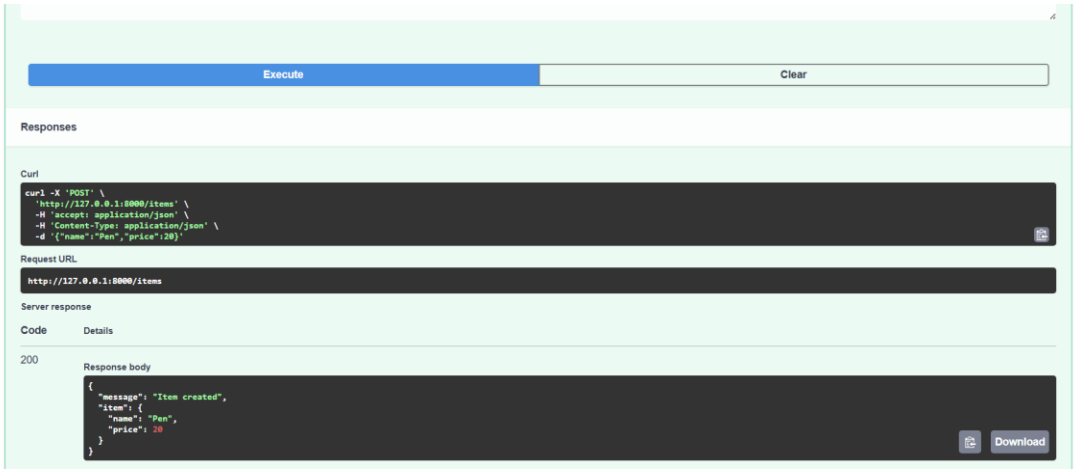
Prompt: Add GET "/items" and POST "/items" endpoints in FastAPI using a list to store items.

Code:

```python
items = []
@app.get("/items")
async def read_items():
    return items
@app.post("/items")
async def create_item(item: dict):
    items.append(item)
    return {"message": "Item created", "item": item}
```

Output:





Explanation: This code defines two endpoints in a FastAPI application to manage a list of items, allowing users to retrieve all items and add new items to the list.

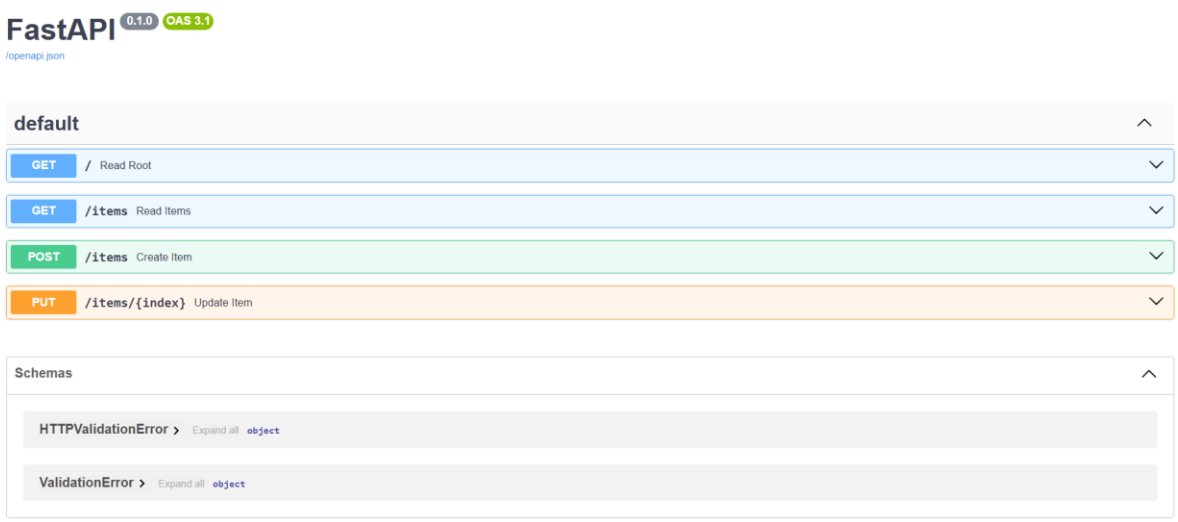
Task 3: Update Existing Item

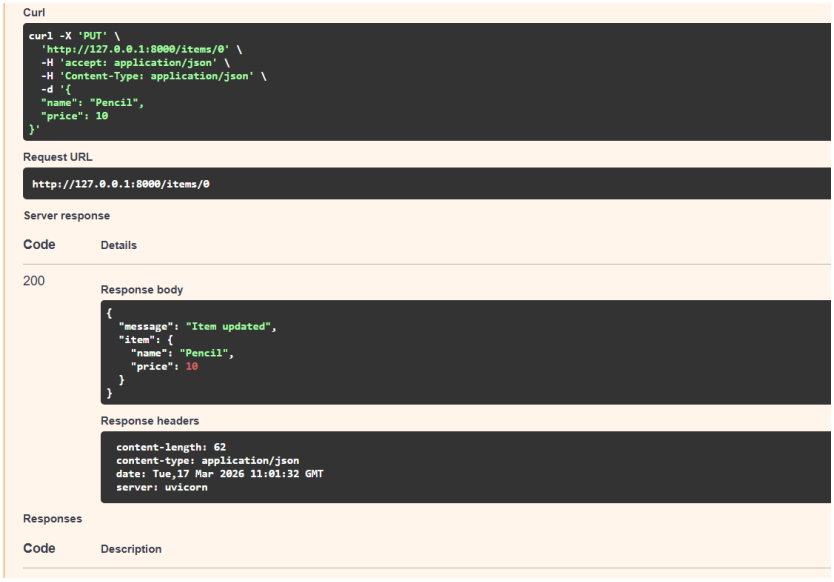
Prompt: Add a PUT "/items/{index}" endpoint to update an item in a list by index.

Code:

```python
@app.put("/items/{index}")
async def update_item(index: int, item: dict):
    if index < 0 or index >= len(items):
        return {"error": "Index out of range"}
    items[index] = item
    return {"message": "Item updated", "item": item}
```

Output:





Explanation: This code adds a PUT endpoint to the FastAPI application that allows users to update an existing item in the list by specifying its index, with error handling for out-of-range indices.

Task 4: Delete Item

Prompt: Add a DELETE "/items/{index}" endpoint to remove an item from a list.

Code:

```python
@app.delete("/items/{index}")
async def delete_item(index: int):
    if index < 0 or index >= len(items):
        return {"error": "Index out of range"}
    removed_item = items.pop(index)
    return {"message": "Item deleted", "item": removed_item}
```

Output:

Curl

```
curl -X 'DELETE' \
  'http://127.0.0.1:8000/items/0' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/items/0
```

Server response

Code	Details
200	Response body

```
{
  "message": "Item deleted",
  "item": {
    "name": "Pen",
    "price": 20
  }
}
```

Response headers

```
content-length: 59
content-type: application/json
date: Tue,17 Mar 2026 11:04:19 GMT
server: uvicorn
```

Responses

Explanation: This code adds a DELETE endpoint to the FastAPI application that allows users to remove an item from the list by specifying its index, with error handling for out-of-range indices.

Task 5: Auto-Generated Documentation

Prompt: Add docstrings to all FastAPI endpoints so they appear in Swagger documentation.

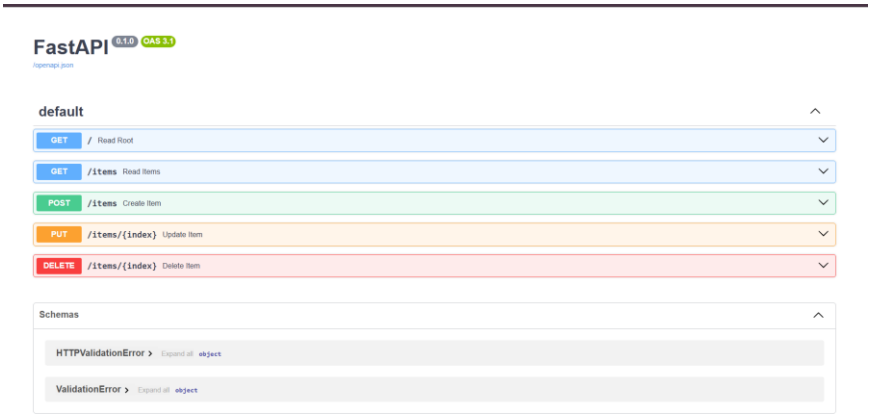
Code:

```python
@app.get("/items")
async def read_items():
    """
    Retrieve all items from the list.
    """
    return items
@app.post("/items")
async def create_item(item: dict):
    """
    Create a new item and add it to the list.
    """
    items.append(item)
    return {"message": "Item created", "item": item}
@app.put("/items/{index}")
async def update_item(index: int, item: dict):
    """
    Update an existing item in the list by its index.
    """
    if index < 0 or index >= len(items):
        return {"error": "Index out of range"}
    items[index] = item
    return {"message": "Item updated", "item": item}
@app.delete("/items/{index}")
async def delete_item(index: int):
    """
    Delete an item from the list by its index.
    """
    if index < 0 or index >= len(items):
        return {"error": "Index out of range"}
    removed_item = items.pop(index)
    return {"message": "Item deleted", "item": removed_item}
```

Open:

- http://127.0.0.1:8000/docs (Swagger UI)

Output:



Explanation: This code adds docstrings to each endpoint in the FastAPI application, providing descriptions that will be displayed in the Swagger UI for better documentation and understanding of each endpoint's functionality.